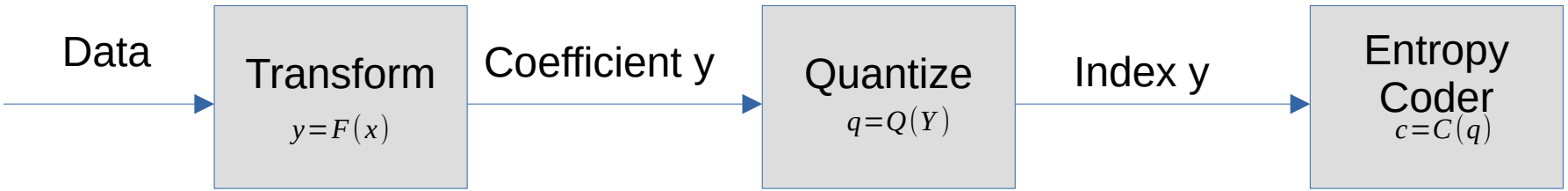
Parallelizing a simple JPEG compression with CUDA

เนื้อหา

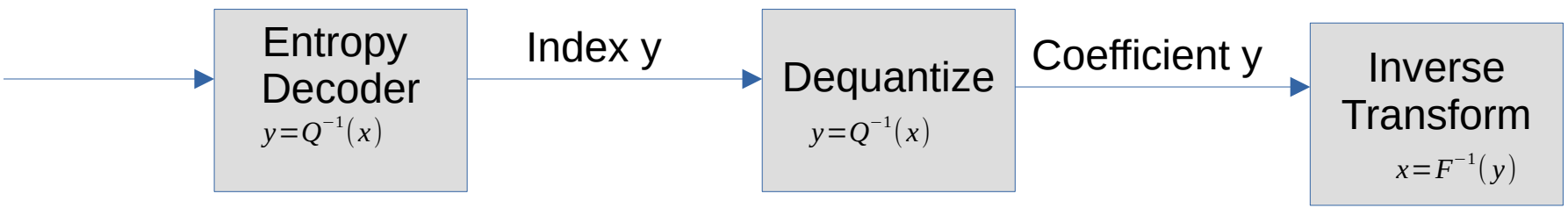
- หลักการคร่าวๆของ Codec .
- อธิบาย JPEG codec และ โครงสร้าง.
- อธิบาย **D**iscrete **C**osine **T**ransform และ การใช้งานใน JPEG codec.
- การออกแบบอัลกอริทึม เดี่ยวของ **DCT**
- การออกแบบอัลกอริทึม ขนานของ **DCT** .
- การนำอัลกอริทึม แบบเดี่ยวของ **DCT** ไปเขียนเป็นโค้ดที่รันบนซีพียู
- การนำอัลกอริทึม แบบขนานของ DCT ไปเขียนเป็นโค้ดที่รันบนจีพียู
- เปรียบเทียบความเร็วที่เพิ่มขึ้น

Codec คืออะไร

Encoding

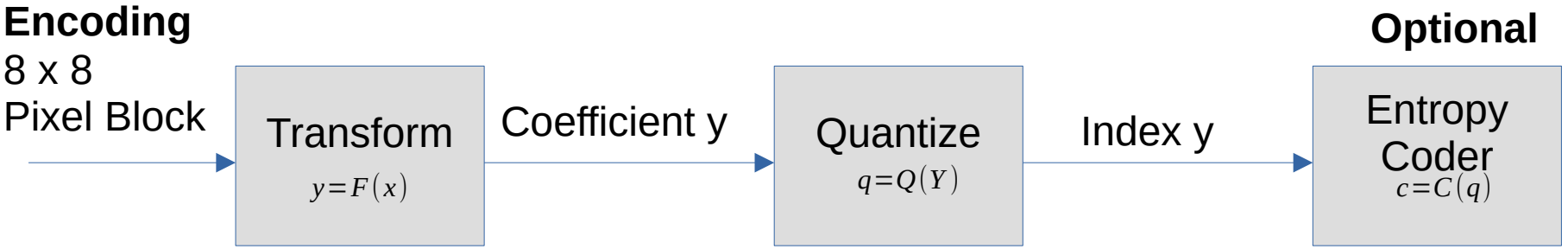


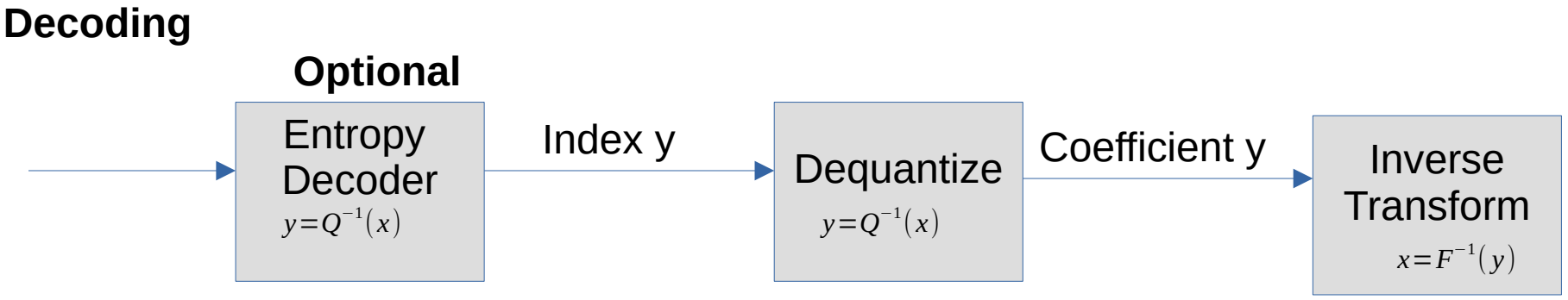
Decoding



Reference

JPEG codec อย่างง่าย

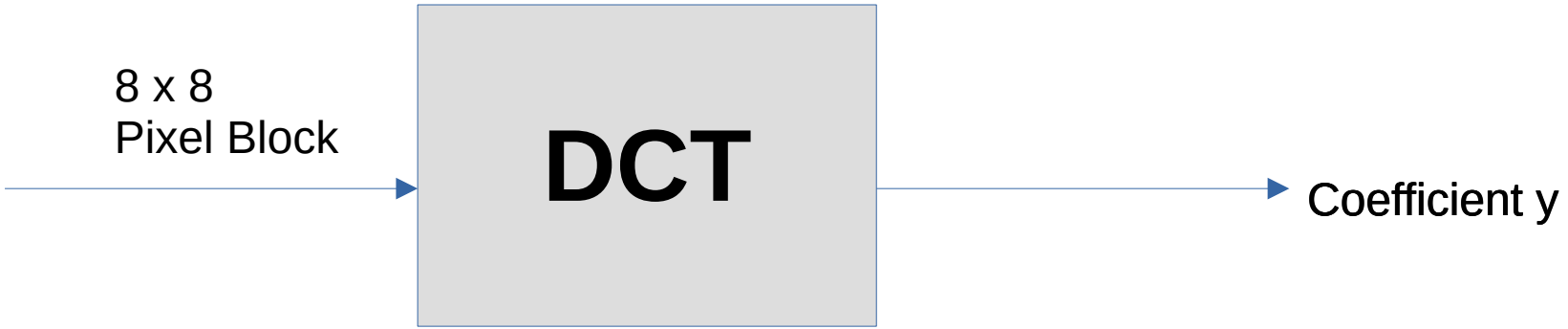




Reference

แล้ว JPEG ใช้ฟังก์ชันอะไรเพื่อ transform?

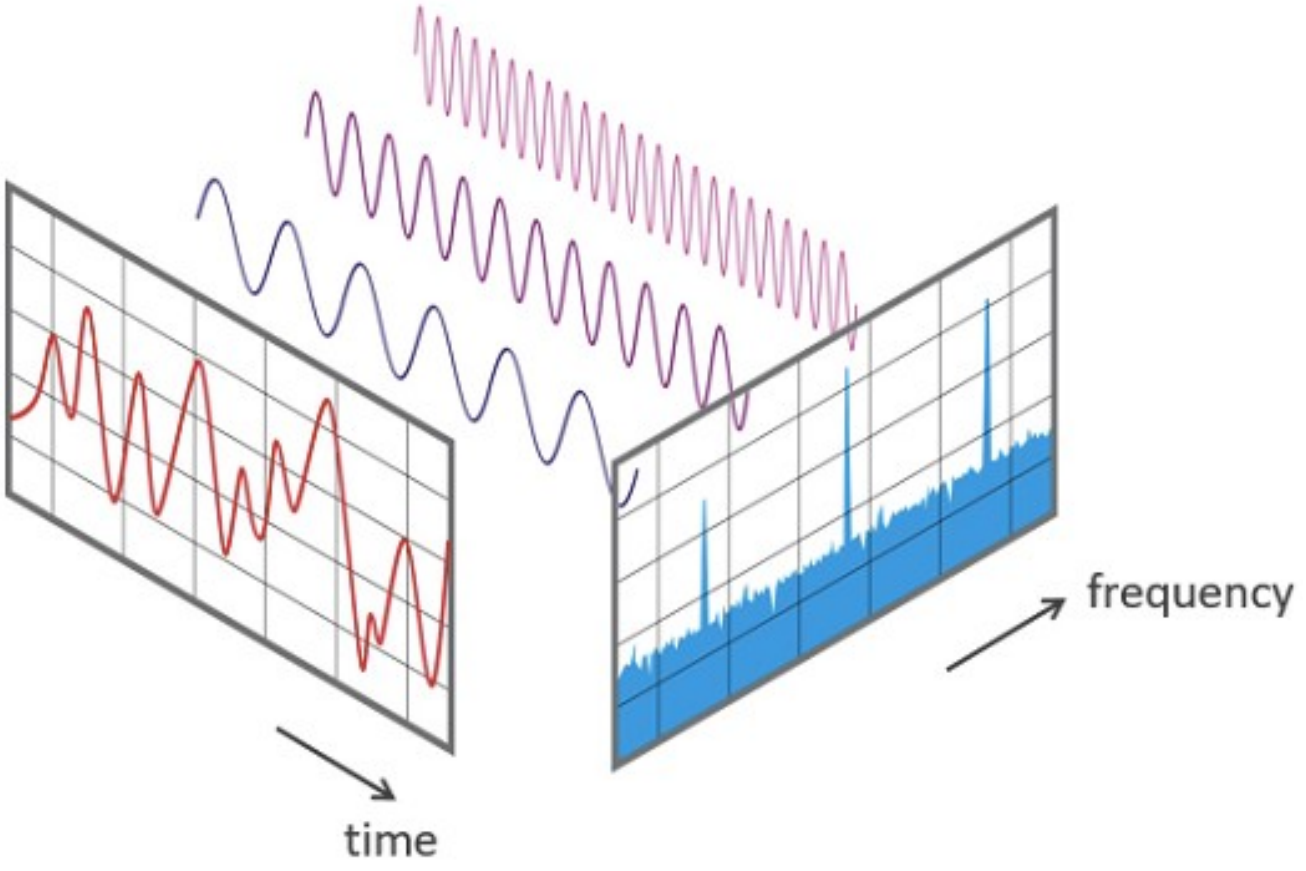
Introducing **D**iscrete **C**osine **T**ransform



A coefficient from the DCT represents the contribution of each cosine waves.

When quantized, some cosine waves will be removed.

Introducing **D**iscrete **C**osine **T**ransform



Introducing **D**iscrete **C**osine **T**ransform

2D Forward Discrete Cosine Transform

$$F_{u,v} = \frac{1}{4} C_v C_u \sum_{y=0}^{N-1} \sum_{x=0}^{N-1} S_{yx} \cos\left(v\pi \frac{2y+1}{2N}\right) \cos\left(u\pi \frac{2x+1}{2N}\right)$$

1D Forward Discrete Cosine Transform

$$F_{u,v} = \frac{1}{2} C_v C_u \sum_{x=0}^{N-1} S_x \cos\left(u\pi \frac{2x+1}{2N}\right)$$

$$C_u = \frac{1}{\sqrt{2}} when\, u = 0 \qquad C_v\, same\, as\, C_u$$

$$1\, else$$

การออกแบบอัลกอริทึม แบบเดี่ยวของ **D**iscrete **C**osine **T**ransform

- Naive Version
- Better Naive Version

(Optional, might not present this)
Discrete **F**ourier **T**ransform related
- Arai-Agui-Nakajima (one of the best)

การออกแบบอัลกอริทึม แบบเดี่ยวของ **D**iscrete **C**osine **T**ransform

Naive Version: Naively follow the definition

$$F_{u,v} = \frac{1}{4} C_v C_u \sum_{y=0}^{N-1} \sum_{x=0}^{N-1} S_{yx} \cos\left(v\pi \frac{2y+1}{2N}\right) \cos\left(u\pi \frac{2x+1}{2N}\right)$$

Better Naive Version: exploiting the separatability of the definition

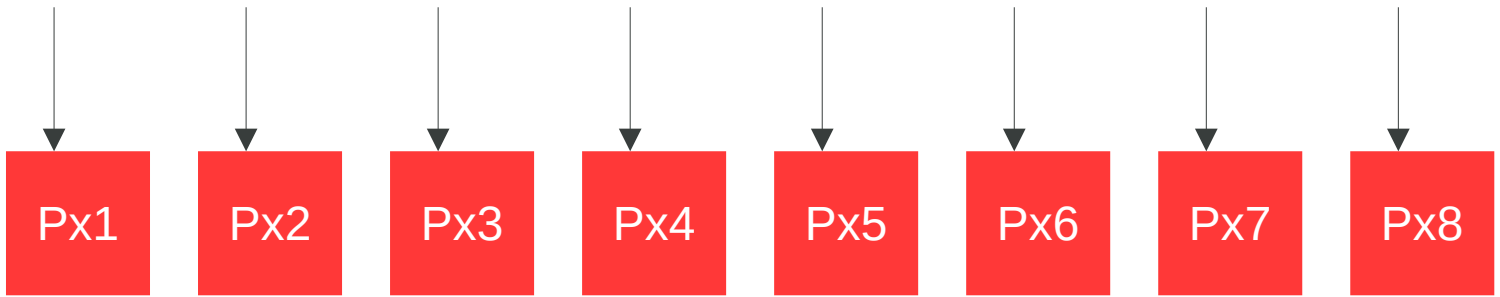
$$F_{u,v} = \frac{1}{4} C_v C_u \sum_{y=0}^{N-1} \cos\left(v\pi \frac{2y+1}{2N}\right) \left[\sum_{x=0}^{N-1} S_{yx} \cos\left(u\pi \frac{2x+1}{2N}\right)\right]$$

(Optional, might not present this)

To get even a better algorithm we need to understand DFT

การออกแบบอัลกอริทึม แบบขนานของ **D**iscrete **C**osine **T**ransform

The Parallel version of the algorithm is the same with sequential but each processor applies its own DCT



การนำอัลกอริทึม แบบเดี่ยวของ **DCT** ไปเขียนเป็นโค้ดที่รันบนซีพียู

Naive Version: Naively follow the definition

```
// Naive Version
void dctNaive(float in[64], float out[64]) {

    const float PI = 3.141592653589793f;
    int u, v, x, y;
    for (v = 0; v < 8; v++)
    for (u = 0; u < 8; u++)
    {

        float sum = 0.0f;
        float Cv = (v == 0) ? 1 / sqrtf(2.0f) : 1.0f;
        float Cu = (u == 0) ? 1 / sqrtf(2.0f) : 1.0f;

        for (y = 0; y < 8; y++)
        for (x = 0; x <8; x++)
        {
            sum += in[y * 8 + x] *
            cosf((2.0f * x + 1.0f) * u * PI / 16.0f) *
            cosf((2.0f * y + 1.0f) * v * PI / 16.0f);
        }

        out[v * 8 + u] = 0.25f * Cv * Cu * sum;
    }
}
```

การนำอัลกอริทึม แบบเดี่ยวของ **DCT** ไปเขียนเป็นโค้ดที่รันบนซีพียู

Better Naive Version: exploiting the separatability of the definition

```
void dct1DPrecompute(float in[8], float out[8]) {

    int u, x;
    for (u = 0; u < 8; u++) {

        float sum = 0.0f;
        float Cu = (u == 0) ? 1 / sqrtf(2.0f) : 1.0f;

        for (x = 0; x < 8; x++) {
            sum += in[x] * COS_TABLE[u][x];
        }

        out[u] = 0.5f * Cu * sum;
    }
}
```

```
void dct2DAs1D(float in[64], float out[64]) {

    int i, j;
    float in1D[8], out1D[8], temp[64];

    // transform row
    for (j = 0; j < 8; j++) {
        for (i = 0; i < 8; i++) in1D[i] = in[j * 8 + i];
        dct1DPrecompute(in1D, out1D);
        for (i = 0; i < 8; i++) temp[j * 8 + i] = out1D[i];
    }

    for (j = 0; j < 8; j++) {
        for (i = 0; i < 8; i++) in1D[i] = temp[i * 8 + j];
        dct1DPrecompute(in1D, out1D);
        for (i = 0; i < 8; i++) out[i * 8 + j] = out1D[i];
    }
}
```

การนำอัลกอริทึม แบบขนานของ DCT ไปเขียนเป็นโค้ดที่รันบนจีพียู

Naive Version: Naively follow the definition

```
#ifdef NAIVE_GPU
float sum = 0.0f;
int i, j;
for (j = 0; j < 8; j++) {

    float Cv = (j == 0) ? 1.0f / sqrtf(2.0f) : 1.0f;
    for (i = 0; i < 8; i++)
    {
        float Cu = (i == 0) ? 1.0f / sqrtf(2.0f) : 1.0f;
        sum += Cu * Cv * tile[j * 8 + i] *
        COS_TABLE_GPU[i * 8 + tx] *
        COS_TABLE_GPU[j * 8 + ty];
    }
}
#endif
```

การนำอัลกอริทึม แบบขนานของ DCT ไปเขียนเป็นโค้ดที่รันบนจีพียู

Better Naive Version: exploiting the separatability of the definition

```
float row_sum = 0.0f;
for (int i = 0; i < 8; i++) {
    float Cu = (i == 0) ? 1.0f / sqrtf(2.0f) : 1.0f;
    row_sum += Cu * tile[ty][i] * COS_TABLE_GPU[i * 8 + tx];
}

__syncthreads();
tile[ty][tx] = row_sum;
__syncthreads();

float col_sum = 0.0f;
for (int j = 0; j < 8; j++) {
    float Cv = (j == 0) ? 1.0f / sqrtf(2.0f) : 1.0f;
    col_sum += Cv * tile[j][tx] * COS_TABLE_GPU[j * 8 + ty];
}
```

เปรียบเทียบความเร็วที่เพิ่มขึ้น

GPU: 3060 CPU: Ryzen 5 5600x

CPU version.

```
terkpc@terkPC:~/Dev/parallel-term-project$ time ./jpeg-codec-cpu

real    0m30.522s
user    0m30.392s
sys     0m0.112s
```

GPU version.

```
terkpc@terkPC:~/Dev/parallel-term-project$ time ./jpeg-codec

real    0m4.894s
user    0m4.463s
sys     0m0.360s
```

เร็วขึ้น 7.5x เท่า